

TP1 – Installation de Python et Premiers Programmes

Objectifs pédagogiques

- Installer et configurer l'environnement Python.
- Écrire et exécuter des programmes simples.
- Manipuler les variables, entrées/sorties, conditions et boucles.
- Appliquer la logique algorithmique à un cas concret.

PARTIE 1 – Travaux guidés

1. Installation de Python

Étapes :

1. Aller sur le site officiel : <https://www.python.org/downloads/>
2. Télécharger la **version 3.12.x** adaptée à votre système.
3. Lors de l'installation, **cocher la case "Add Python to PATH"** avant de cliquer sur *Install Now*.
4. Une fois l'installation terminée, ouvrir un terminal et taper :
5. `python --version`

Exemple de réponse attendue :
Python 3.12.3

6. Installer un éditeur de code:
 - **Thonny** (idéal pour les débutants)
7. Créer un nouveau fichier nommé `bonjour.py` et y écrire :
8. `print("Bonjour, bienvenue dans le cours de Python !")`

Explication :

`print()` permet d'afficher un texte à l'écran.
C'est la première instruction à tester après installation.

2. Variables et entrées utilisateur

```
nom = input("Entrez votre nom : ")
age = int(input("Entrez votre âge : "))
print("Bonjour", nom, "vous avez", age, "ans.")
```

Explication :

- `input()` lit une saisie clavier.
- `int()` convertit la saisie en entier.

- Les variables `nom` et `age` stockent les données saisies.

Remarque :

Sans la conversion `int()`, la variable `age` serait une chaîne de caractères (texte).

3. Conditions simples

```
note = float(input("Entrez votre note : "))
if note >= 10:
    print("Admis")
else:
    print("Ajourné")
```

Explication :

- `if` vérifie une condition logique.
- `else` s'exécute si la condition est fausse.
- L'indentation (4 espaces) est obligatoire pour délimiter les blocs.

4. Conditions multiples avec `elif`

```
note = float(input("Entrez votre note : "))

if note >= 16:
    print("Excellent")
elif note >= 14:
    print("Bien")
elif note >= 10:
    print("Passable")
else:
    print("Ajourné")
```

Explication :

- `elif` signifie “sinon si”.
- Les conditions sont testées dans l'ordre.
- Dès qu'une condition est vraie, les suivantes sont ignorées.

5. Boucles

Exemple 1 : Boucle `for`

```
for i in range(1, 6):
    print("Itération :", i)
```

Explication :

`range(1, 6)` génère les nombres 1 à 5.
La boucle répète les instructions 5 fois.

Exemple 2 : Boucle `while`

```
n = 0
```

```
while n < 3:
    print("Essai n°", n + 1)
    n += 1
```

Explication :

La boucle `while` continue tant que la condition `n < 3` est vraie.
On incrémente `n` pour éviter une boucle infinie.

6. Mini-application : Somme de plusieurs nombres

```
n = int(input("Combien de nombres voulez-vous additionner ? "))
somme = 0
for i in range(n):
    x = float(input("Entrez un nombre : "))
    somme += x
print("La somme totale est :", somme)
```

Explication :

- `somme` est un accumulateur.
- Chaque valeur saisie est ajoutée à la somme totale.
- `somme += x` est équivalent à `somme = somme + x`.

PARTIE 2 – Exercice d’évaluation

Sujet : Analyse personnalisée des notes d’un groupe

Nom du fichier : `analyse_classe.py`

Énoncé étudiant

Écrire un programme qui permet d’analyser les notes d’un groupe d’étudiants.
Le programme doit :

1. Demander le **nombre d’étudiants**.
2. Pour chaque étudiant, saisir son **nom** et sa **note**.
3. Calculer et afficher :
 - la **moyenne de la classe**,
 - le **nombre d’étudiants admis** ($\text{note} \geq 10$),
 - la **meilleure note** et le **nom de l’étudiant correspondant**,
 - une **appréciation** selon la moyenne :
 - “Très bon groupe” si $\text{moyenne} \geq 14$
 - “Groupe moyen” si $10 \leq \text{moyenne} < 14$
 - “Doit s’améliorer” sinon.

Remarque : Il faut utiliser `round()` pour afficher les résultats finaux.